FleetOps — Guia de implantação e operação

Documentação de portfólio do sistema genérico de gestão de frota.
Gerado em 09/11/2026

Guia de implantação — FleetOps

Campo	Valor
Versão	1.0
Data	26 de outubro de 2026
Classificação	Uso interno
Procedimento Docker	Extraído de `compose.yaml`, Dockerfiles, `README-DOCKER.md` e `docker-entrypoint.sh`
TLS no Compose	**Não implementado** (recomendado à frente do Nginx)

Complementa README-DOCKER.md.

Requisitos (produção Compose)

- Debian com Docker Engine 24+ e `docker compose`.
- Porta `HTTP_PORT` livre (padrão 80).
- Rede de saída no **build** (PyPI, npm). Em runtime, FIPE e Google Fonts são opcionais.
- Disco para volumes `pgdata`, `media`, `staticfiles`.

Preparar o servidor (recomendado, não automatizado neste repo)

- Usuário no grupo `docker`.
- Diretório da aplicação (exemplo genérico: `/opt/fleetops` — ajuste à política local).
- Copiar o projeto **sem** `.env` de outro ambiente.
- `[ -f .env ] || cp .env.example .env` e preencher chaves.

Subir

```
docker compose up -d --build
docker compose ps
docker compose exec backend python manage.py createsuperuser
```

Ordem: `db` healthy → `backend` (espera PG, `migrate`, `collectstatic`, Gunicorn) → `web`. `rotina` sobe com `SKIP_MIGRATE=1` após db e backend healthy.

Este fluxo está no Compose e no entrypoint. A execução em servidor de produção **não** foi ensaiada neste repositório.

Há units em `deploy/systemd/` como alternativa **sem** o container `rotina`. Não os habilite se o serviço Compose `rotina` já estiver no ar (duplicaria a rotina). Ajuste o caminho `/opt/fleetops` à instalação real.

Pós-implantação

```
curl -sI "http://127.0.0.1:${HTTP_PORT:-80}/" | head -5
curl -sI "http://127.0.0.1:${HTTP_PORT:-80}/admin/login/" | head -5
curl -s -o /dev/null -w "%{http_code}\n" -X POST \
  -H "Content-Type: application/json" -d '{}' \
  "http://127.0.0.1:${HTTP_PORT:-80}/api/usuarios/login/"
docker compose exec backend python manage.py check
docker compose exec backend python manage.py showmigrations | tail -20
```

O login JWT é **POST** (`CustomTokenObtainPairView`). Esperado com corpo vazio: **400**. GET no mesmo URL: **405**. Credenciais inválidas em POST: **401**. 502 = proxy/backend. Esta checagem **não** valida TLS.

Acessos: `/` SPA, `/api/`, `/admin/`, `/media/`.

Atualização

```
# preserve .env
```

```
docker compose up -d --build
```

O backend reaplica migrate. Backup antes: backup-restauracao.md.

Rollback (recomendado)

Não há ferramenta de rollback no repositório. Procedimento proposto (pendente de ensaio):

- Manter artefato/código da versão anterior.
- Restaurar dump PostgreSQL e tarball de mídia **testados**.
- `docker compose up -d --build` na versão anterior.
- Não usar `down -v`.

Desenvolvimento versus produção

	Dev	Compose (prod documentado)
DEBUG	True (default settings)	False
API	:8000 direto	Nginx `/api`
SPA	Vite :5173	estáticos no Nginx
DB	settings ou Supabase no example do BACKEND	`db:5432`, SSL disable
Rotina	comando manual	serviço `rotina`
CORS LAN	ligado se DEBUG	`CORS_ALLOW_LAN=False`

Falhas no boot

Sintoma	Causa provável
backend unhealthy	migrate lento, `ALLOWED_HOSTS` sem localhost, Gunicorn não sobe
502	backend ainda não escuta
DisallowedHost	host público ausente em `ALLOWED_HOSTS`
CSRF Admin	origem sem esquema em `CSRF_TRUSTED_ORIGINS`

Runbook operacional — FleetOps

Campo	Valor
Versão	1.0
Data	26 de outubro de 2026
Classificação	Uso interno
Risco	Não usar `docker compose down -v` nem `flush`

Pré-condição: diretório do Compose e permissão Docker.

1. Estado dos serviços

```
docker compose ps
docker compose exec backend python manage.py check
```

Esperado: `db`, `backend`, `rotina`, `web` Up (backend/web healthy).

2. Logs

```
docker compose logs -f --tail=100
docker compose logs -f backend
docker compose logs -f rotina
docker compose logs -f web
docker compose logs -f db
```

Loggers de aplicação: `reservas.operacoes`, `vistorias.desempenho` (stdout).

3. Frontend não carrega

- `curl -sI http://127.0.0.1:${HTTP_PORT:-80}/`
- Logs `web`.
- Confirmar build da imagem (SPA está na imagem, não em volume de código).

4. API 502 / 401 inesperado

- Logs `backend`.
- Healthcheck: `docker compose exec backend python -c "import urllib.request; urllib.request.urlopen(urllib.request.Request('http://127.0.0.1:8000/admin/login/', headers={'Host':'localhost'}), timeout=5)"`
- 401 em rotas autenticadas é normal sem Bearer.

5. Banco

```
docker compose exec db pg_isready -U frota -d frota
```

Ajuste usuário/banco ao `.env` (`DB_USER`, `DB_NAME`).

Não publique a porta 5432 no host (Compose atual não publica).

6. Reinício seguro

```
docker compose stop
docker compose start
# ou
docker compose up -d
```

Reinício do `backend` dispara migrate de novo (idempotente se já aplicado).

7. Disco

```
df -h
docker system df
```

Volumes críticos: `pgdata`, `media`. Limpeza `docker system prune` **não** é rotina (risco de apagar imagens ainda úteis; não remove volumes nomeados sem `-a --volumes`).

8. Rotina operacional parada

Sintoma: reservas aprovadas sem check-in não cancelam após a meia-noite do dia seguinte.

```
docker compose logs rotina --tail=50
docker compose exec backend python manage.py rotina_operacional
```

O comando avulso no backend executa **uma** passagem. O serviço `rotina` é o loop.

9. Escalação

- Reproduzir horário (fuso America/Sao_Paulo).
- Guardar `docker compose logs` (sem colar `.env`).
- Verificar auditoria (`/auditoria`) se a ação deveria ter gerado registro.
- Envolver TI para SMTP, DNS, TLS e disco.

10. Confirmar recuperação

- `docker compose ps` healthy
- Login na SPA
- `POST /api/usuarios/login/` com `{}` responde 400 (GET nesse URL responde 405)
- Uma reserva de teste em homologação, nunca dados de produção improvisados